SIEM-Based Brute-Force Detection & Investigation

A hands-on Splunk project using simulated authentication logs

Revised documentation — screenshots placed in workflow order

Platform	Splunk Enterprise
Input file	sample_authentication_logs.csv
Index	main
Scope	Data import → SPL searches → event timeline → finding

1. Project overview

I used Splunk Enterprise to practise a beginner SOC workflow with a small, simulated authentication-log CSV. The goal was to import the data, look for repeated failed logins, compare failures with successes, and inspect event order before writing a cautious finding.

What I wanted to practise

• Importing and searching a CSV in Splunk.

• Counting failed logins by IP address and username.

• Grouping failures into five-minute intervals.

• Comparing failed and successful authentication events.

• Using a timeline without overstating the result.

2. Start Splunk and open Search & Reporting

After installing Splunk Enterprise, I opened the local web interface. The home page showed available apps, including Search & Reporting.

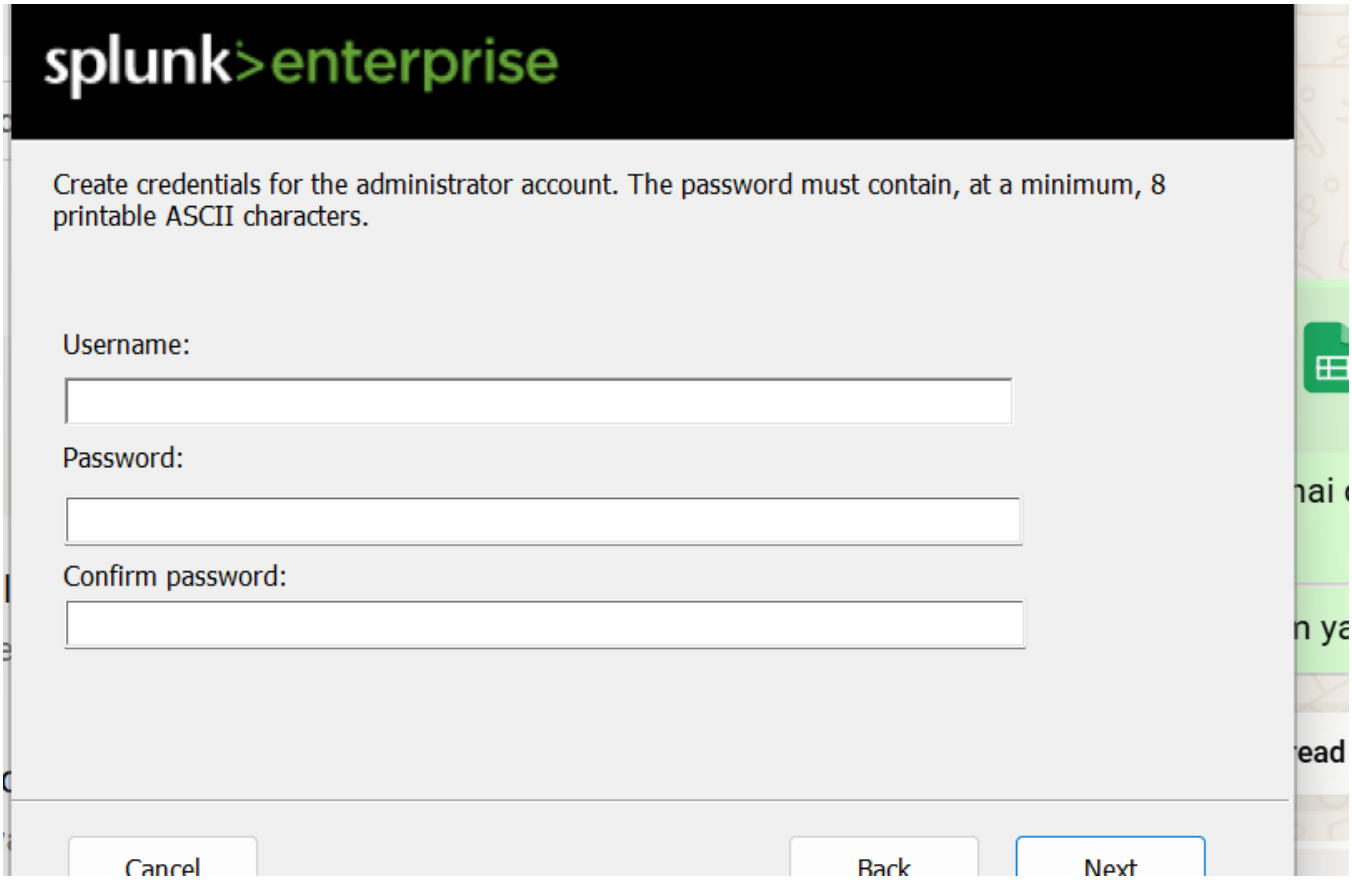


Figure 1. Splunk Enterprise first-run administrator credentials screen.

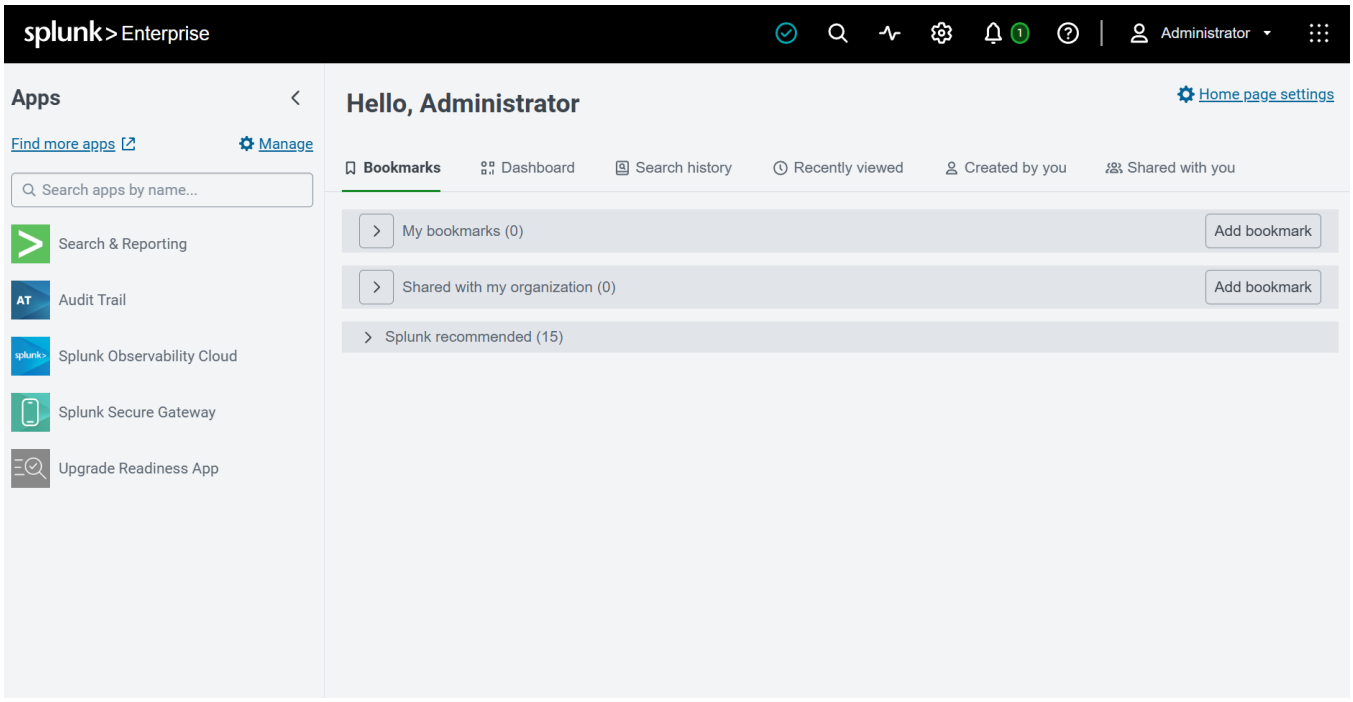


Figure 2. Splunk Enterprise home page after signing in.

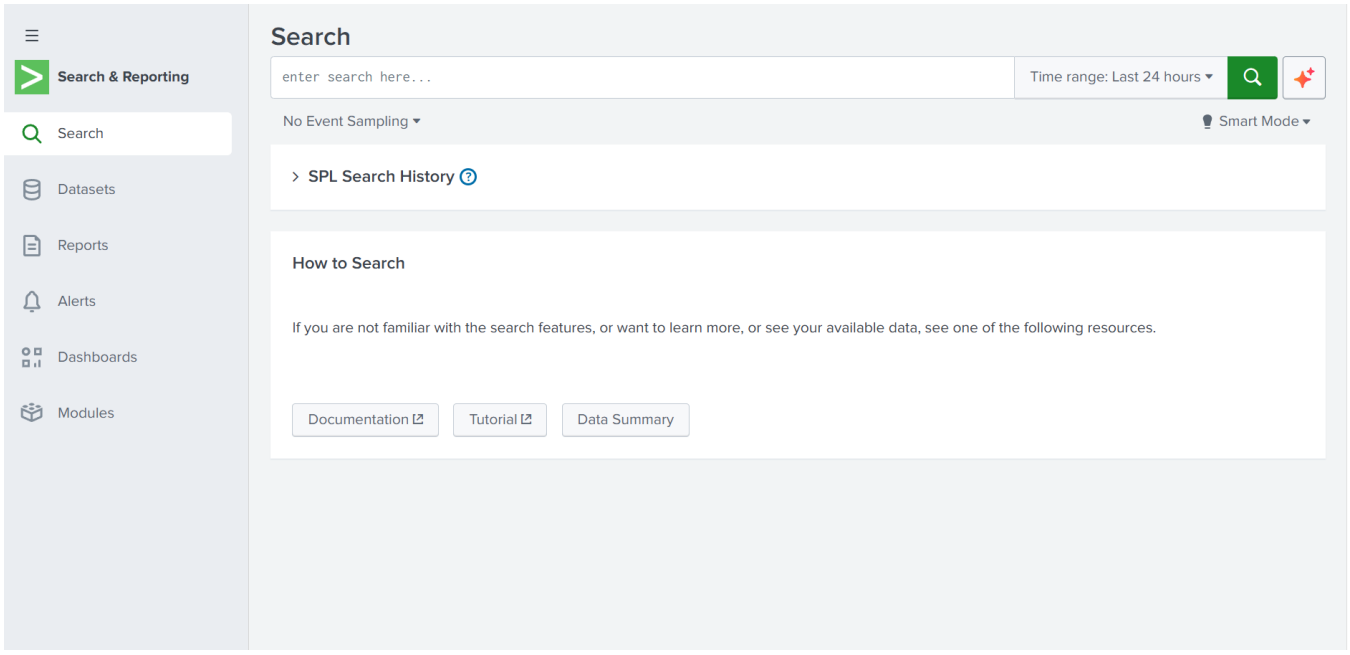


Figure 3. Search & Reporting app, where SPL searches are entered.

3. Upload the sample authentication logs

I used Add Data to upload sample_authentication_logs.csv and continued through the source type and input-settings steps.

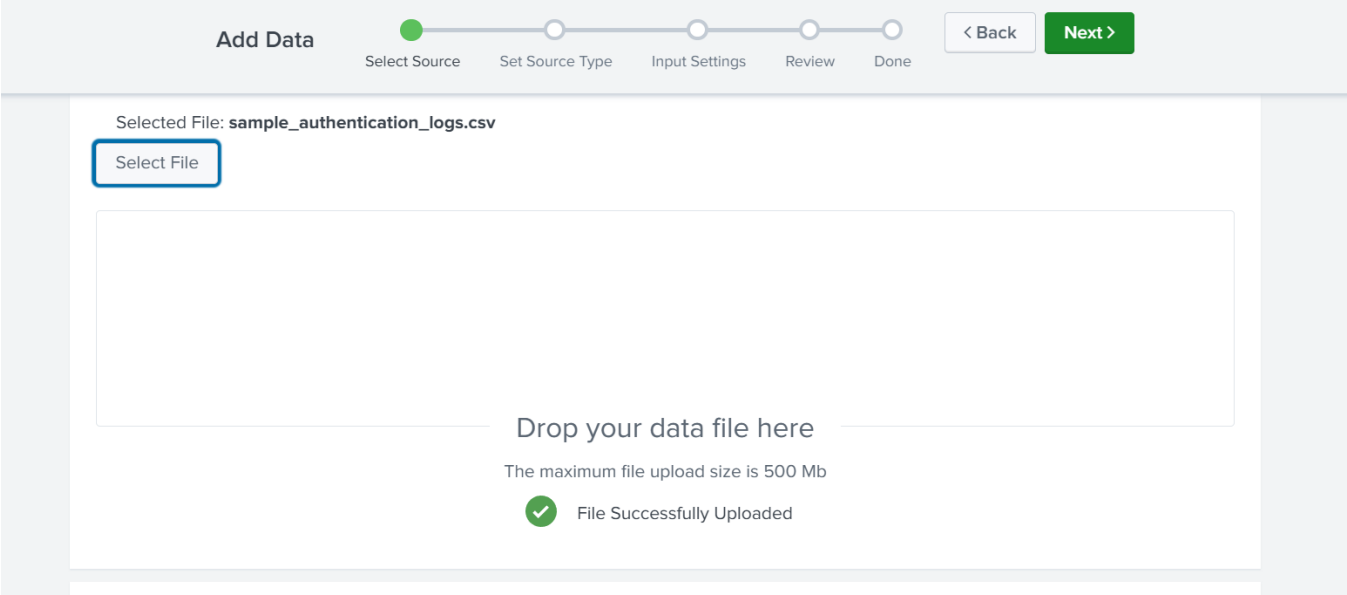


Figure 4. CSV file selected and successfully uploaded.

I kept the host value shown by Splunk and selected the main index. The review screen confirmed the file, source type, host, and index.

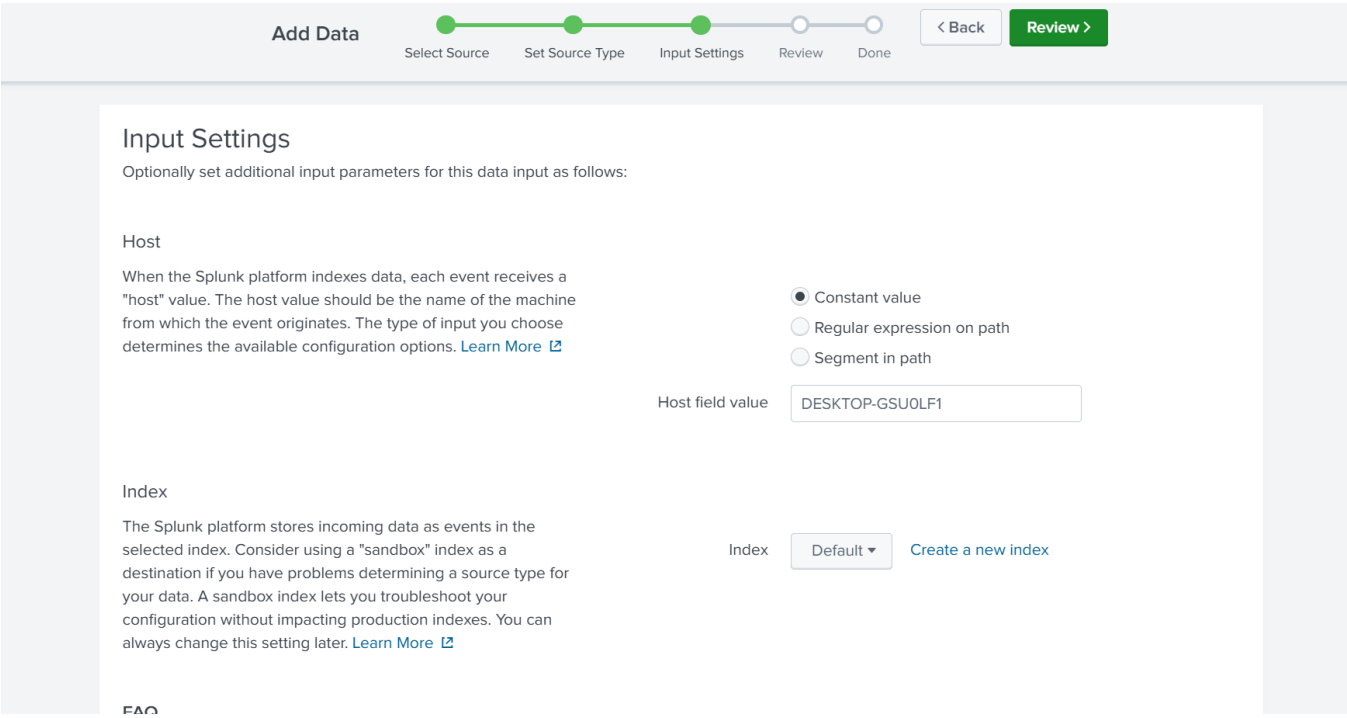


Figure 5. Input Settings before reviewing the data input.



Figure 6. Review screen confirming the upload configuration.

4. Confirm that the events are searchable

The search index=main returned 20 events, confirming the file had been ingested.

```
index=main
```

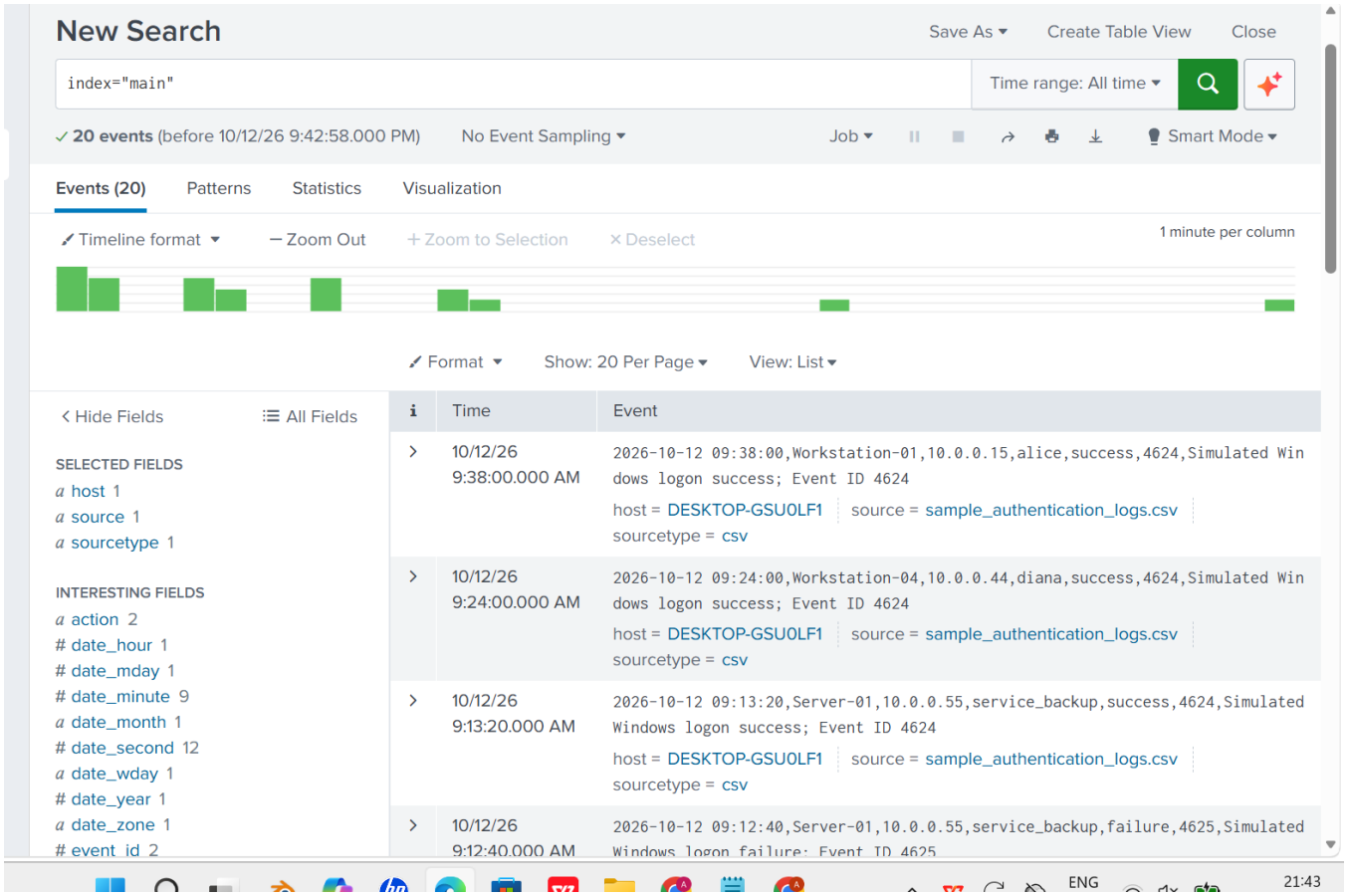


Figure 7. Initial event search showing imported authentication records.

5. Count failed logins by source IP and username

I used a simple query to find which IP-and-account combinations had the most failures.

```
index=main action=failure
| stats count by source_ip username
| sort - count
```

action=failure filters unsuccessful logins. stats counts each source_ip/username pair, and sort - count puts the largest count first.

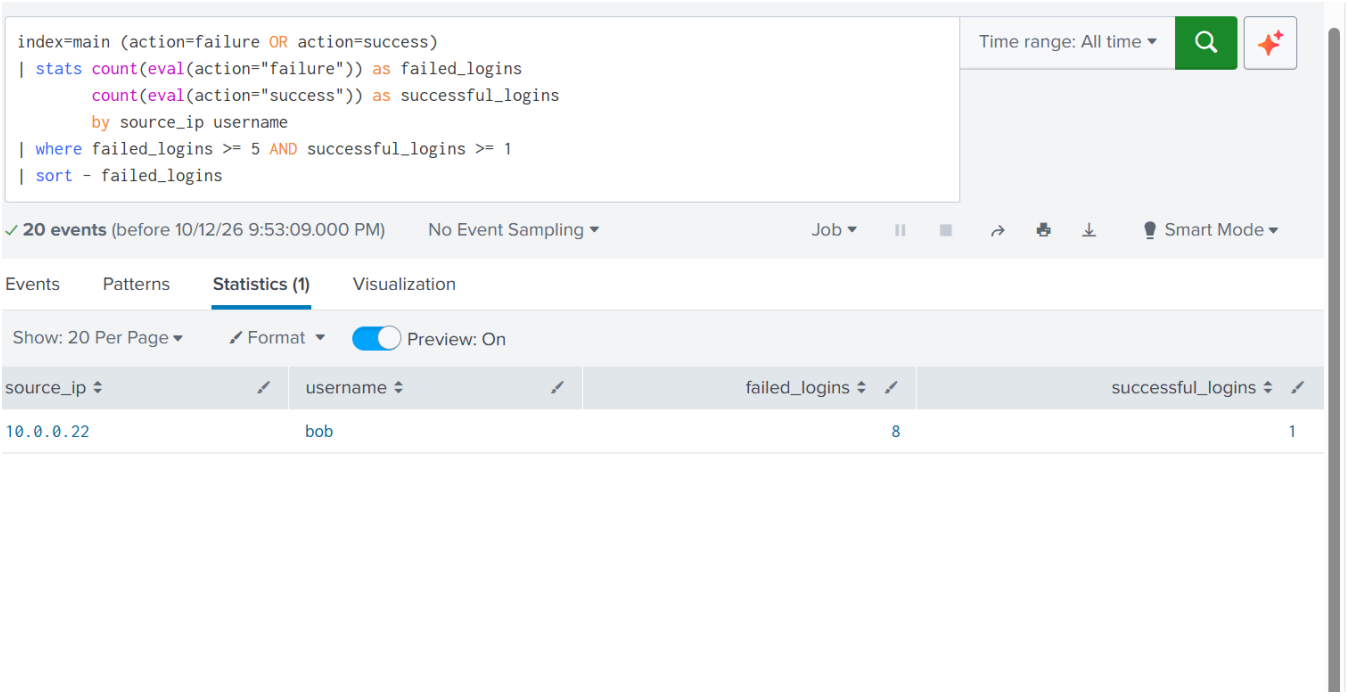


Figure 8. Failed-login totals grouped by source IP and username.

The results showed 15 failures. The largest group was 10.0.0.22 targeting bob, with eight failures. A total count does not show whether attempts happened close together.

6. Look for five or more failures in a five-minute bucket

```
index=main action=failure
| eval event_time=strptime(timestamp,"%Y-%m-%d %H:%M:%S")
| bin event_time span=5m
| stats count by event_time source_ip username
| where count >= 5
| sort - count
```

The first output showed 1791775800, a Unix timestamp. It is a numeric time representation, not a broken date.

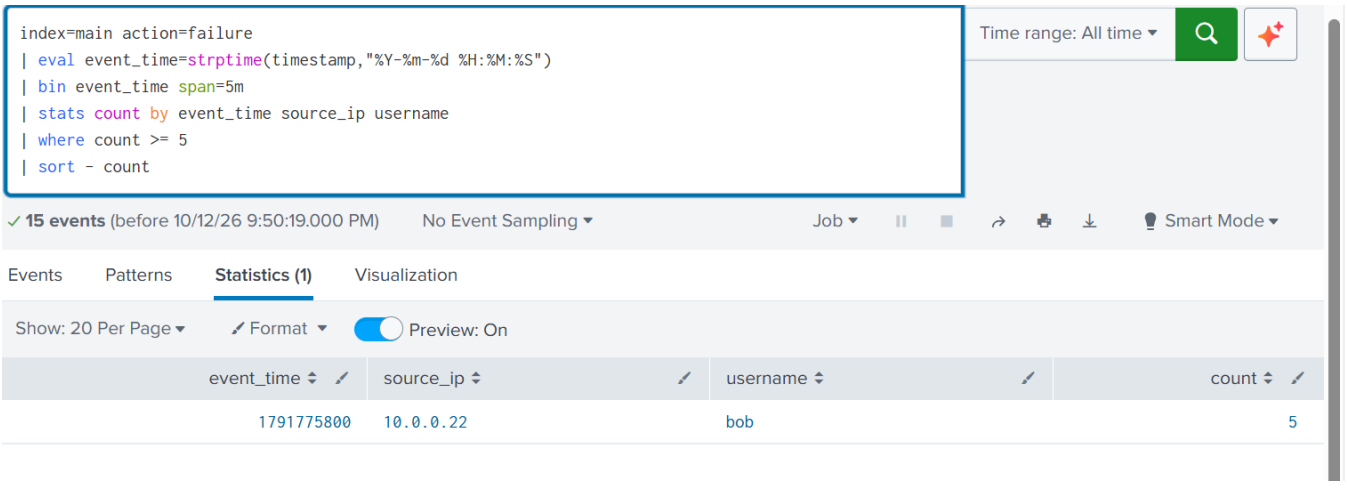


Figure 9. First time-bucket result with event_time displayed as a Unix timestamp.

I then formatted the timestamp as readable text.

```
index=main action=failure
| eval event_time=strptime(timestamp,"%Y-%m-%d %H:%M:%S")
| bin event_time span=5m
| eval event_time=strftime(event_time,"%Y-%m-%d %H:%M:%S")
| stats count by event_time source_ip username
| where count >= 5
| sort - count
```

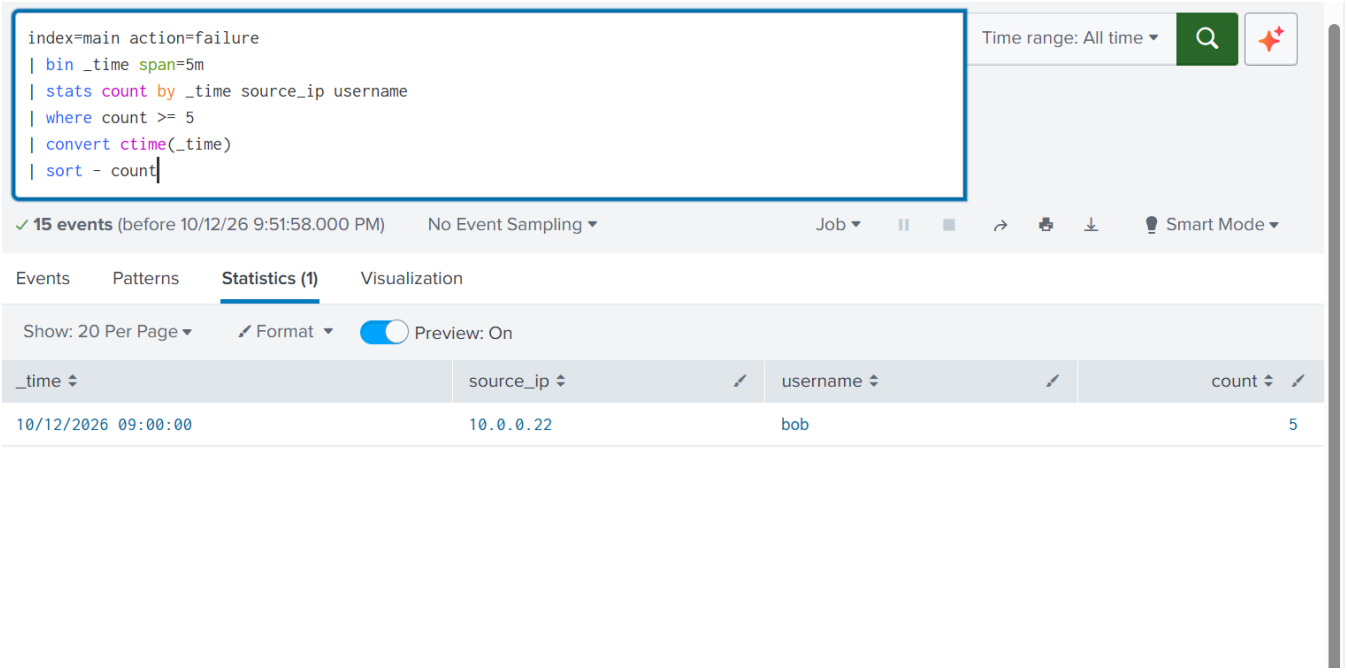


Figure 10. Five-minute bucket shown with a readable timestamp.

The result showed five failures for 10.0.0.22 / bob in the bucket beginning at 09:00. The bucket time marks the interval start. Five failures is a practice threshold, not a universal rule.

7. Compare failed and successful logins

```
index=main (action=failure OR action=success)
| stats count(eval(action="failure")) as failed_logins
        count(eval(action="success")) as successful_logins
        by source_ip username
| where failed_logins >= 5 AND successful_logins >= 1
| sort - failed_logins
```

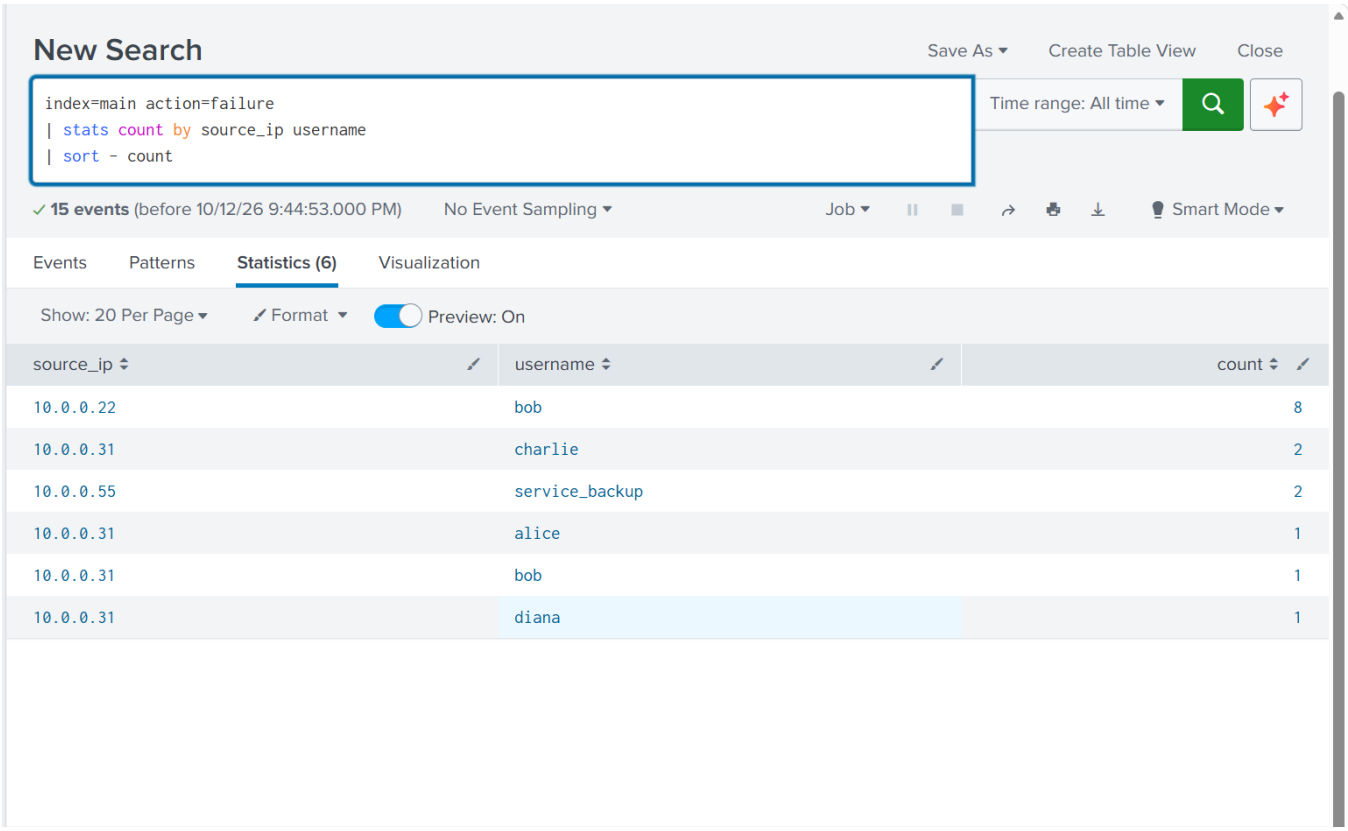
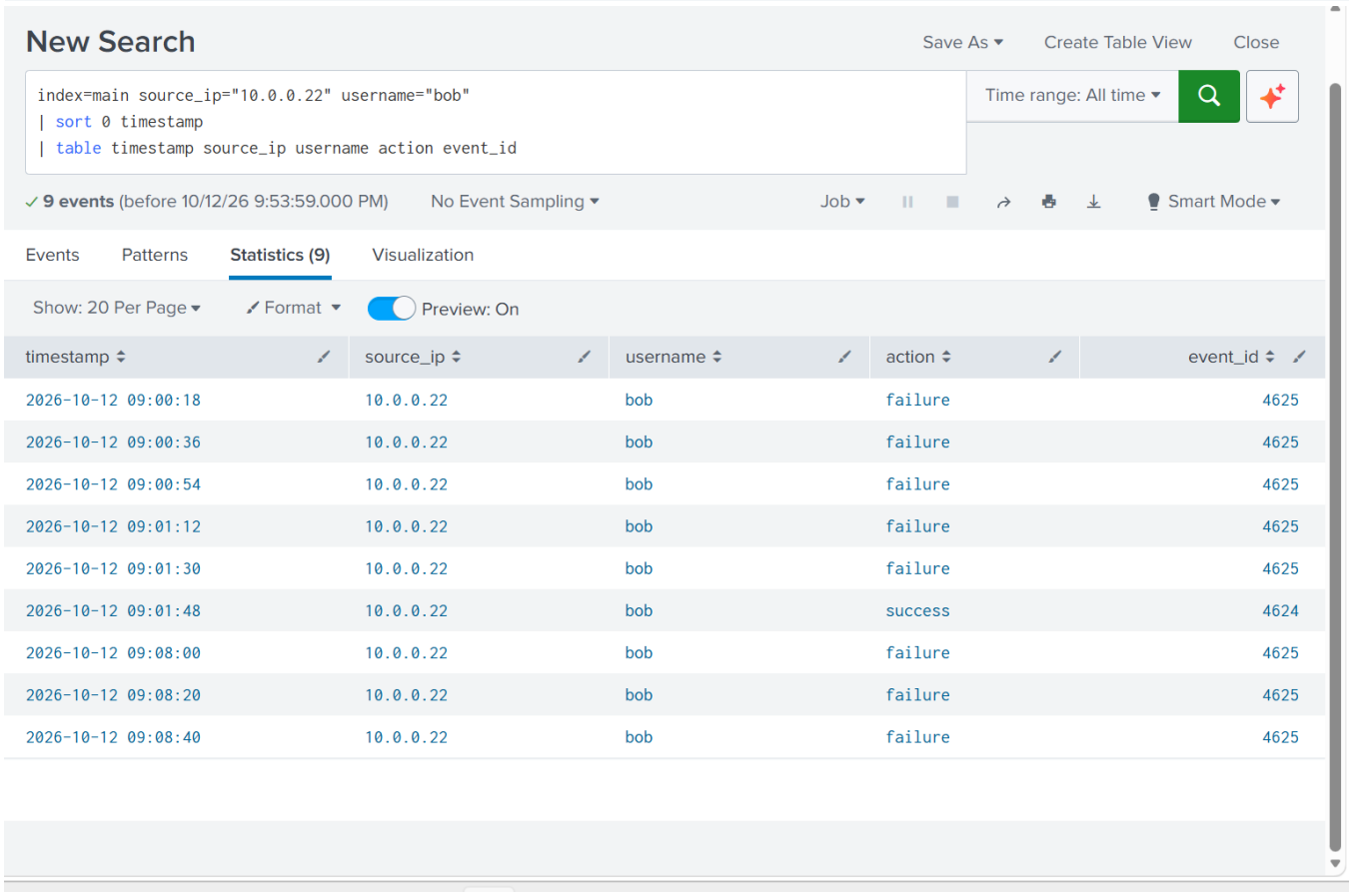


Figure 11. Aggregate comparison showing eight failures and one success for 10.0.0.22 / bob.

This aggregate identifies a candidate but does not prove event order, so I reviewed the individual timestamps.

8. Inspect the individual event timeline

```
index=main source_ip="10.0.0.22" username="bob"
| sort 0 timestamp
| table timestamp source_ip username action event_id
```



timestamp	source_ip	username	action	event_id
2026-10-12 09:00:18	10.0.0.22	bob	failure	4625
2026-10-12 09:00:36	10.0.0.22	bob	failure	4625
2026-10-12 09:00:54	10.0.0.22	bob	failure	4625
2026-10-12 09:01:12	10.0.0.22	bob	failure	4625
2026-10-12 09:01:30	10.0.0.22	bob	failure	4625
2026-10-12 09:01:48	10.0.0.22	bob	success	4624
2026-10-12 09:08:00	10.0.0.22	bob	failure	4625
2026-10-12 09:08:20	10.0.0.22	bob	failure	4625
2026-10-12 09:08:40	10.0.0.22	bob	failure	4625

Figure 12. Individual events sorted chronologically for 10.0.0.22 and bob.

The screenshot shows five failures between 09:00:18 and 09:01:30, followed by a successful login at 09:01:48. Three more failures appear between 09:08:00 and 09:08:40. The success came 18 seconds after the fifth failure.

This is suspicious enough to investigate, but does not prove a brute-force attack or account compromise. In a real environment, I would validate the source IP, inspect the successful event's logon type, and correlate other account or endpoint activity.

9. Initial investigation finding

Repeated failed logins followed by successful authentication

Source IP: 10.0.0.22

Account: bob

Evidence: five failures from 09:00:18 to 09:01:30, a success at 09:01:48, then three further failures around 09:08.

Assessment: suspicious authentication pattern in simulated data. Account compromise is not confirmed.

Next checks: validate the source IP; inspect logon type and surrounding account activity; correlate with other endpoint or network logs.

10. What I learned and limitations

- A total count and a time-window count answer different questions.
- A time bucket labels an interval, not each event's exact time.
- Aggregate counts identify candidates, but a timeline establishes event order.
- Suspicious activity is not proof of compromise.
- The CSV is synthetic; an alert and dashboard are not shown as completed.

Appendix: Queries used

A. Confirm ingestion

```
index=main
```

B. Failed-login totals

```
index=main action=failure
| stats count by source_ip username
| sort - count
```

C. Five-minute grouping

```
index=main action=failure
| eval event_time=strptime(timestamp,"%Y-%m-%d %H:%M:%S")
| bin event_time span=5m
| stats count by event_time source_ip username
| where count >= 5
| sort - count
```

D. Compare outcomes

```
index=main (action=failure OR action=success)
| stats count(eval(action="failure")) as failed_logins
        count(eval(action="success")) as successful_logins
        by source_ip username
| where failed_logins >= 5 AND successful_logins >= 1
| sort - failed_logins
```

E. Timeline

```
index=main source_ip="10.0.0.22" username="bob"
| sort 0 timestamp
| table timestamp source_ip username action event_id
```